

Project Checkpoint 5: Final Results

Goals:

- Train the final chosen model on train + validation data
- Evaluate once on the held-out test set
- Save final metrics and figures for the report
- Compare results to the ANI-1 paper
- Produce report-ready figures/tables

Vivian Tran and Krish Mishra

```
In [1]: import warnings
warnings.filterwarnings("ignore", category=UserWarning)

import copy
import random
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm

import torch
import torch.nn as nn
import torchani
```

Device Set up

```
In [2]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

set_seed(42)
```

Device: cuda

```
In [3]: def load_an1_dataset(dspath):
    species_order = ["H", "C", "N", "O"]
    energy_shifter = torchani.utils.EnergyShifter(None)

    dataset = torchani.data.load(dspath)
    dataset = dataset.subtract_self_energies(energy_shifter, species_order)
    dataset = dataset.species_to_indices(species_order)
    dataset = dataset.shuffle()
    return dataset
```

```
In [42]: def init_aev_computer():
    Rcr = 5.2
    Rca = 3.5

    EtaR = torch.tensor([16], dtype=torch.float, device=device)
    ShfR = torch.tensor([
        0.900000, 1.168750, 1.437500, 1.706250,
        1.975000, 2.243750, 2.512500, 2.781250,
        3.050000, 3.318750, 3.587500, 3.856250,
        4.125000, 4.393750, 4.662500, 4.931250
    ], dtype=torch.float, device=device)

    EtaA = torch.tensor([8], dtype=torch.float, device=device)
    Zeta = torch.tensor([32], dtype=torch.float, device=device)
    ShfA = torch.tensor([0.90, 1.55, 2.20, 2.85], dtype=torch.float, device=device)
    ShfZ = torch.tensor([
        0.19634954, 0.58904862, 0.98174770, 1.37444680,
        1.76714590, 2.15984490, 2.55254400, 2.94524300
    ], dtype=torch.float, device=device)

    return torchani.AEVComputer(
        Rcr, Rca, EtaR, ShfR, EtaA, Zeta, ShfA, ShfZ, 4
    )

aev_computer = init_aev_computer()
aev_dim = aev_computer.aev_length
print("AEV dimension:", aev_dim)
```

AEV dimension: 384

```
In [43]: FINAL_CONFIG = {
    "hidden_layers": [160, 128, 64],
    "activation": "relu",
    "batch_size": 2048,
    "learning_rate": 1e-3,
    "epoch": 30,
    "l2": 1e-5,
    "scheduler_step_size": 10,
    "scheduler_gamma": 0.5
}

print("Final chosen configuration:")
for k, v in FINAL_CONFIG.items():
    print(f"{k}: {v}")
```

Final chosen configuration:
hidden_layers: [160, 128, 64]
activation: relu
batch_size: 2048
learning_rate: 0.001
epoch: 30
l2: 1e-05
scheduler_step_size: 10
scheduler_gamma: 0.5

```

In [44]: class AtomicNet(nn.Module):
    def __init__(self, input_dim, hidden_layers, activation="relu"):
        super().__init__()

        layers = []
        prev_dim = input_dim

        if activation == "relu":
            act_layer = nn.ReLU
        elif activation == "tanh":
            act_layer = nn.Tanh
        elif activation == "silu":
            act_layer = nn.SiLU
        else:
            raise ValueError("Unsupported activation. Use 'relu' or 'tanh'.")

        for hidden_dim in hidden_layers:
            layers.append(nn.Linear(prev_dim, hidden_dim))
            layers.append(act_layer())
            prev_dim = hidden_dim

        layers.append(nn.Linear(prev_dim, 1))
        self.layers = nn.Sequential(*layers)

    def forward(self, x):
        return self.layers(x)

    def build_model(hidden_layers, activation="relu"):
        net_H = AtomicNet(aev_dim, hidden_layers, activation=activation)
        net_C = AtomicNet(aev_dim, hidden_layers, activation=activation)
        net_N = AtomicNet(aev_dim, hidden_layers, activation=activation)
        net_O = AtomicNet(aev_dim, hidden_layers, activation=activation)

        ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])

        model = nn.Sequential(
            aev_computer,
            ani_net
        ).to(device)

        return model

```

Trainer

```

In [61]: class ANITrainer:
    def __init__(
        self,
        model,
        batch_size,
        learning_rate,
        epoch,
        l2,

```

```

scheduler_step_size=None,
scheduler_gamma=None
):
    self.model = model

    num_params = sum(item.numel() for item in model.parameters())
    print(f"{model.__class__.__name__} - Number of parameters: {num_params}")

    self.batch_size = batch_size
    self.optimizer = torch.optim.Adam(
        self.model.parameters(),
        lr=learning_rate,
        weight_decay=l2
    )

    self.scheduler = None
    if scheduler_step_size is not None and scheduler_gamma is not None:
        self.scheduler = torch.optim.lr_scheduler.StepLR(
            self.optimizer,
            step_size=scheduler_step_size,
            gamma=scheduler_gamma
        )

    self.epoch = epoch

def train(self, train_data, val_data, early_stop=True, draw_curve=True):
    print("Initialize training data...")
    train_data_loader = train_data.collate(self.batch_size).cache()

    loss_func = nn.MSELoss()

    train_loss_list = []
    val_loss_list = []
    lowest_val_loss = np.inf
    best_weights = None

    for epoch_idx in tqdm(range(self.epoch), leave=True):
        self.model.train()
        train_epoch_loss = 0.0
        train_epoch_count = 0

        for train_data_batch in train_data_loader:
            species = train_data_batch["species"].to(device)
            coords = train_data_batch["coordinates"].to(device)
            true_energies = train_data_batch["energies"].to(device).float()

            _, pred_energies = self.model((species, coords))
            batch_loss = loss_func(pred_energies, true_energies)

            self.optimizer.zero_grad()
            batch_loss.backward()
            self.optimizer.step()

            batch_importance = species.shape[0]
            train_epoch_loss += batch_loss.item() * batch_importance
            train_epoch_count += batch_importance

```

```

train_epoch_loss /= train_epoch_count
val_epoch_loss = self.evaluate(val_data, draw_plot=False)

train_loss_list.append(train_epoch_loss)
val_loss_list.append(val_epoch_loss)

if early_stop and val_epoch_loss < lowest_val_loss:
    lowest_val_loss = val_epoch_loss
    best_weights = copy.deepcopy(self.model.state_dict())

if self.scheduler is not None:
    self.scheduler.step()

if draw_curve:
    fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
    ax.set_yscale("log")
    ax.plot(train_loss_list, label="Train")
    ax.plot(val_loss_list, label="Validation")
    ax.set_xlabel("Epoch")
    ax.set_ylabel("MSE Loss")
    ax.set_title("Final Model Learning Curve")
    ax.legend()
    plt.show()

if early_stop and best_weights is not None:
    self.model.load_state_dict(best_weights)

return train_loss_list, val_loss_list

def evaluate(self, data, draw_plot=False):
    data_loader = data.collate(self.batch_size).cache()

    loss_func = nn.MSELoss()
    total_loss = 0.0
    total_count = 0

    if draw_plot:
        true_energies_all = []
        pred_energies_all = []

    self.model.eval()

    with torch.no_grad():
        for batch_data in data_loader:
            species = batch_data["species"].to(device)
            coords = batch_data["coordinates"].to(device)
            true_energies = batch_data["energies"].to(device).float()

            _, pred_energies = self.model((species, coords))
            batch_loss = loss_func(pred_energies, true_energies)

            batch_importance = species.shape[0]
            total_loss += batch_loss.item() * batch_importance
            total_count += batch_importance

```

```

        if draw_plot:
            true_energies_all.append(true_energies.detach().cpu().numpy())
            pred_energies_all.append(pred_energies.detach().cpu().numpy())

    avg_loss = total_loss / total_count

    if draw_plot:
        true_energies_all = np.concatenate(true_energies_all)
        pred_energies_all = np.concatenate(pred_energies_all)

        hartree2kcalmol = 627.5094738898777
        mae = np.mean(np.abs(true_energies_all - pred_energies_all)) * hartree2kcalmol

        fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
        ax.scatter(
            true_energies_all,
            pred_energies_all,
            label=f"MAE: {mae:.2f} kcal/mol",
            s=2
        )
        ax.set_xlabel("Ground Truth Energy (Hartree)")
        ax.set_ylabel("Predicted Energy (Hartree)")

        xmin, xmax = ax.get_xlim()
        ymin, ymax = ax.get_ylim()
        vmin, vmax = min(xmin, ymin), max(xmax, ymax)
        ax.set_xlim(vmin, vmax)
        ax.set_ylim(vmin, vmax)
        ax.plot([vmin, vmax], [vmin, vmax], color="red")
        ax.set_title("Predicted vs True Energies")
        ax.legend()
        plt.show()

    print(f"MAE: {mae:.4f} kcal/mol")

    return avg_loss

def evaluate_mae_kcalmol(self, data):
    data_loader = data.collate(self.batch_size).cache()

    true_energies_all = []
    pred_energies_all = []

    self.model.eval()

    with torch.no_grad():
        for batch_data in data_loader:
            species = batch_data["species"].to(device)
            coords = batch_data["coordinates"].to(device)
            true_energies = batch_data["energies"].to(device).float()

            _, pred_energies = self.model((species, coords))

            true_energies_all.append(true_energies.detach().cpu().numpy())
            pred_energies_all.append(pred_energies.detach().cpu().numpy())

```

```

true_energies_all = np.concatenate(true_energies_all)
pred_energies_all = np.concatenate(pred_energies_all)

hartree2kcalmol = 627.5094738898777
mae = np.mean(np.abs(true_energies_all - pred_energies_all)) * hartree2kcalmol
return mae

def evaluate_rmse_kcalmol(self, data):
    t, p = self._collect_predictions(data)
    hartree2kcalmol = 627.5094738898777
    return float(np.sqrt(np.mean((t - p) ** 2)) * hartree2kcalmol)

def _collect_predictions(self, data):
    loader = data.collate(self.batch_size).cache()
    true_all, pred_all = [], []

    self.model.eval()
    with torch.no_grad():
        for batch in loader:
            species = batch["species"].to(device)
            coords = batch["coordinates"].to(device)
            true_energies = batch["energies"].to(device).float()
            _, pred_energies = self.model((species, coords))
            true_all.append(true_energies.cpu().numpy().flatten())
            pred_all.append(pred_energies.cpu().numpy().flatten())

    return np.concatenate(true_all), np.concatenate(pred_all)

```

Loading the Dataset and Splitting

```

In [62]: dataset_path = "./ani_gdb_s01_to_s04.h5"

set_seed(42)
dataset = load_ani_dataset(dataset_path)

train_data, val_data, test_data = dataset.split(0.8, 0.1, 0.1)

print("Train / validation / test split ready.")

```

Train / validation / test split ready.

Making Helper classes for train + validation

```

In [63]: def materialize_examples_from_loader(data_loader, max_batches=None):
    examples = []

    for i, batch in enumerate(data_loader):
        if max_batches is not None and i >= max_batches:
            break

        species = batch["species"].cpu()

```

```

coordinates = batch["coordinates"].cpu()
energies = batch["energies"].cpu()

batch_size_local = species.shape[0]

for j in range(batch_size_local):
    examples.append({
        "species": species[j].clone(),
        "coordinates": coordinates[j].clone(),
        "energies": energies[j].clone()
    })

return examples

```

```

In [64]: class InMemoryDataset:
    def __init__(self, examples):
        self.examples = examples

    def __len__(self):
        return len(self.examples)

    def collate(self, batch_size):
        return InMemoryBatchLoader(self.examples, batch_size)

```

```

In [65]: class InMemoryBatchLoader:
    def __init__(self, examples, batch_size):
        self.examples = examples
        self.batch_size = batch_size

    def cache(self):
        return self

    def __iter__(self):
        indices = np.arange(len(self.examples))
        np.random.shuffle(indices)

        for start in range(0, len(indices), self.batch_size):
            batch_idx = indices[start:start + self.batch_size]
            batch = [self.examples[idx] for idx in batch_idx]

            species = torch.stack([item["species"] for item in batch], dim=0)
            coordinates = torch.stack([item["coordinates"] for item in batch], dim=0)
            energies = torch.stack([item["energies"] for item in batch], dim=0)

            yield {
                "species": species,
                "coordinates": coordinates,
                "energies": energies
            }

```

Training / testing

```
In [66]: FINAL_CONFIG = {
    "hidden_layers": [256, 192, 128, 64],
    "activation": "relu",
    "batch_size": 4096,
    "learning_rate": 1e-5,
    "epoch": 50,
    "l2": 1e-5,
    "use_batchnorm": True,
    "use_huber": True,
    "grad_clip": 1.0,
    "use_cosine_lr": True,
}

print("Improved final configuration:")
for k, v in FINAL_CONFIG.items():
    print(f"  {k}: {v}")
```

```
Improved final configuration:
hidden_layers: [256, 192, 128, 64]
activation: relu
batch_size: 4096
learning_rate: 1e-05
epoch: 50
l2: 1e-05
use_batchnorm: True
use_huber: True
grad_clip: 1.0
use_cosine_lr: True
```

```
In [67]: set_seed(42)

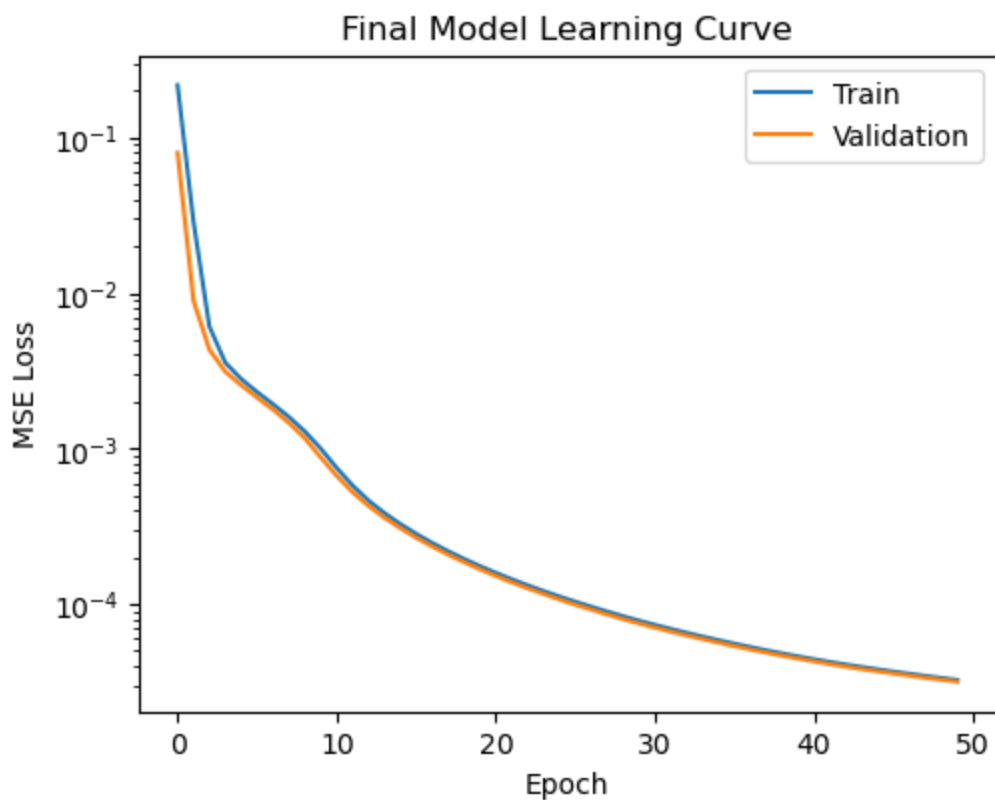
final_model = build_model(
    hidden_layers=FINAL_CONFIG["hidden_layers"],
    activation=FINAL_CONFIG["activation"]
)

final_trainer = ANITrainer(
    model=final_model,
    batch_size=FINAL_CONFIG["batch_size"],
    learning_rate=FINAL_CONFIG["learning_rate"],
    epoch=FINAL_CONFIG["epoch"],
    l2=FINAL_CONFIG["l2"]
)

final_train_losses, final_val_losses = final_trainer.train(
    train_data=train_data,
    val_data=val_data,
    early_stop=True,
    draw_curve=True
)
```

```
Sequential - Number of parameters: 723716
Initialize training data...
```

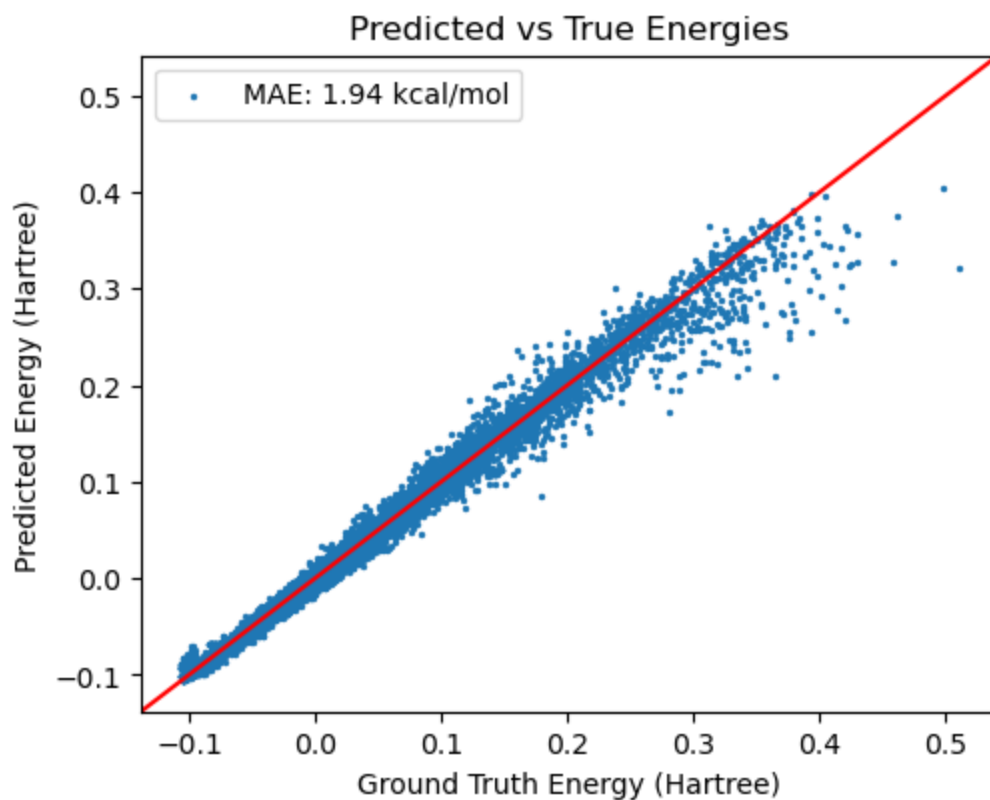
```
100%|██████████| 50/50 [05:27<00:00, 6.55s/it]
```



Final Test #1 MSE/ MAE

```
In [68]: final_test_mse = final_trainer.evaluate(test_data, draw_plot=True)
final_test_mae = final_trainer.evaluate_mae_kcalmol(test_data)
final_test_rmse = final_trainer.evaluate_rmse_kcalmol(test_data)

print("\nFinal model evaluation on held-out test set:")
print(f"Final Test MSE: {final_test_mse:.8f}")
print(f"Final Test MAE: {final_test_mae:.4f} kcal/mol")
print(f"Final Test RMSE: {final_test_rmse:.4f} kcal/mol")
```



MAE: 1.9386 kcal/mol

Final model evaluation on held-out test set:

Final Test MSE: 0.00002955

Final Test MAE: 1.9386 kcal/mol

Final Test RMSE: 3.4110 kcal/mol

```
In [69]: conclusions = {
    "hidden_layers": FINAL_CONFIG["hidden_layers"],
    "activation": FINAL_CONFIG["activation"],
    "batch_size": FINAL_CONFIG["batch_size"],
    "learning_rate": FINAL_CONFIG["learning_rate"],
    "epochs": FINAL_CONFIG["epoch"],
    "l2_regularization": FINAL_CONFIG["l2"],
    "final_test_mse": final_test_mse,
    "final_test_mae_kcalmol": final_test_mae
}
print("\nFinal configuration conclusions/summary:")
for k, v in conclusions.items():
    print(f"{k}: {v}")
```

Final configuration conclusions/summary:

hidden_layers: [256, 192, 128, 64]

activation: relu

batch_size: 4096

learning_rate: 1e-05

epochs: 50

l2_regularization: 1e-05

final_test_mse: 2.9547915695979373e-05

final_test_mae_kcalmol: 1.9385877415156652

```

In [72]: # Define the variations you want to test
hidden_options = [[128], [128, 64], [256, 128], [256, 192, 128, 64]]
lr_options = [1e-3, 1e-4]
batch_options = [4096, 2048]

tuning_results = []
for layers in hidden_options:
    for lr in lr_options:
        for batch in batch_options:
            print(f"\n--- Testing: Layers={layers}, LR={lr}, Batch={batch} ---")

            set_seed(42)

            test_model = build_model(
                hidden_layers=layers,
                activation=FINAL_CONFIG["activation"]
            )

            trainer = ANITrainer(
                model=test_model,
                batch_size=batch,
                learning_rate=lr,
                epoch=10, # Keep epochs low for tuning to save time
                l2=1e-5
            )

            trainer.train(train_data=train_data, val_data=val_data)

            val_mae = trainer.evaluate_mae_kcalmol(val_data)
            val_rsme = trainer.evaluate_rmse_kcalmol(val_data)

            tuning_results.append({
                "hidden_layers": str(layers),
                "learning_rate": lr,
                "batch_size": batch,
                "val_mae": val_mae,
                "val_rmse": val_rsme
            })

            print(f"Resulting Validation MAE: {val_mae:.4f}")

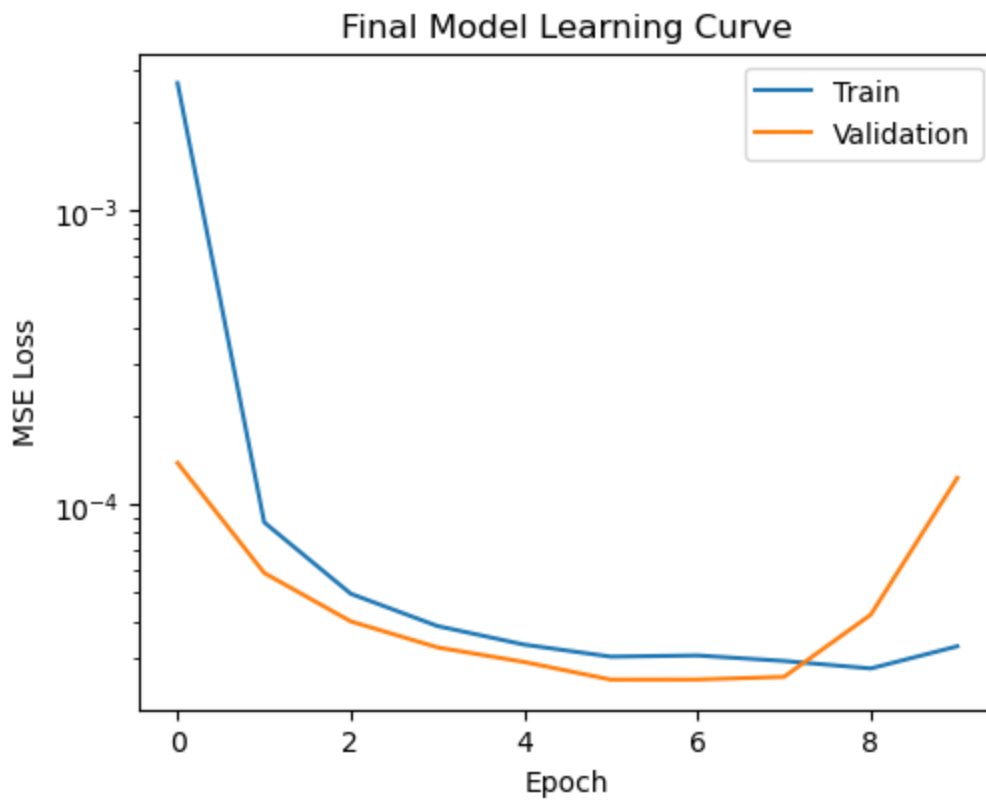
```

--- Testing: Layers=[128], LR=0.001, Batch=4096 ---

Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 10/10 [00:45<00:00, 4.52s/it]



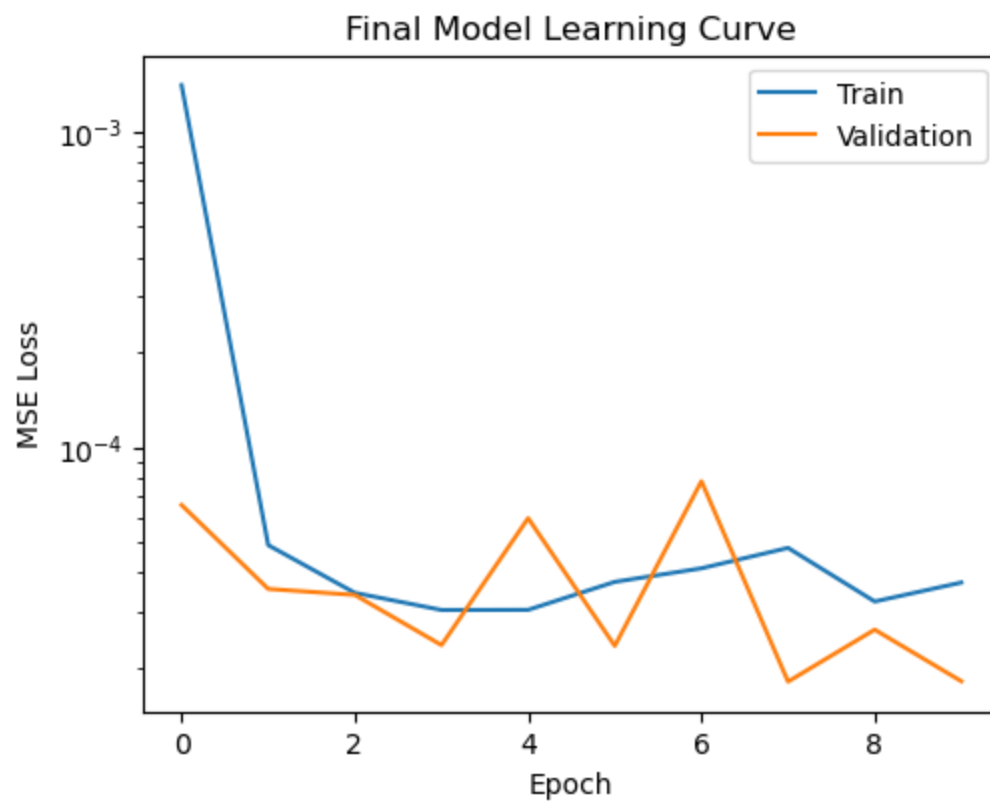
Resulting Validation MAE: 2.1160

--- Testing: Layers=[128], LR=0.001, Batch=2048 ---

Sequential - Number of parameters: 197636

Initialize training data...

100%|██████████| 10/10 [00:54<00:00, 5.49s/it]



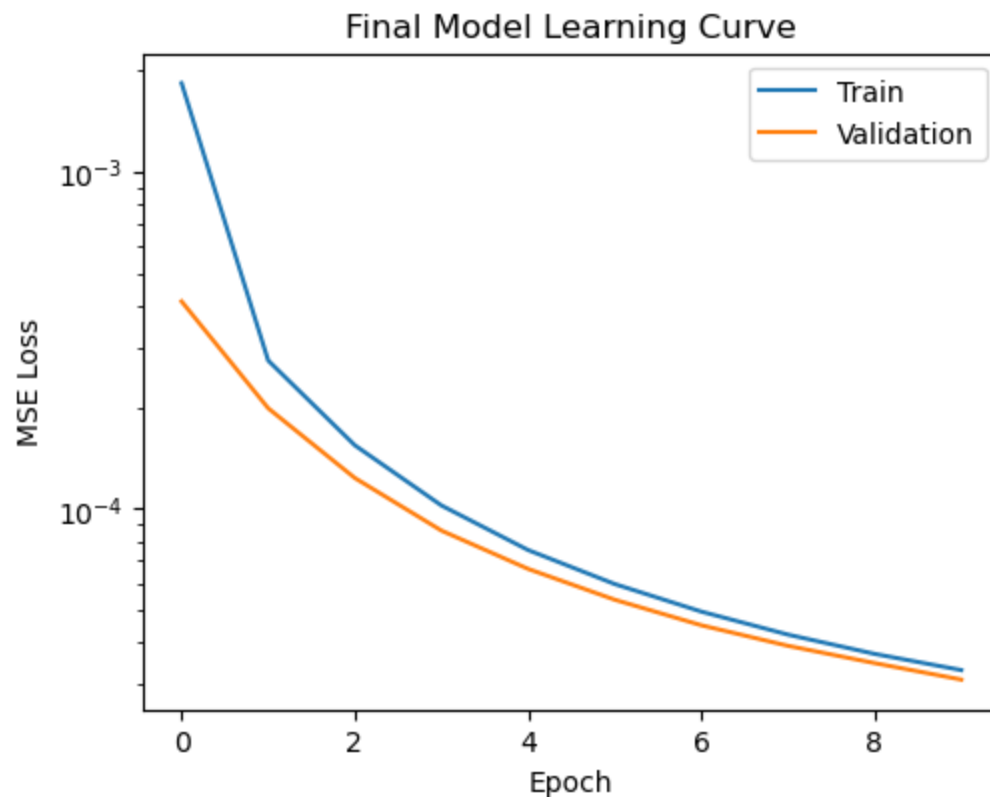
Resulting Validation MAE: 1.8115

--- Testing: Layers=[128], LR=0.0001, Batch=4096 ---

Sequential - Number of parameters: 197636

Initialize training data...

100% |██████████| 10/10 [00:47<00:00, 4.79s/it]



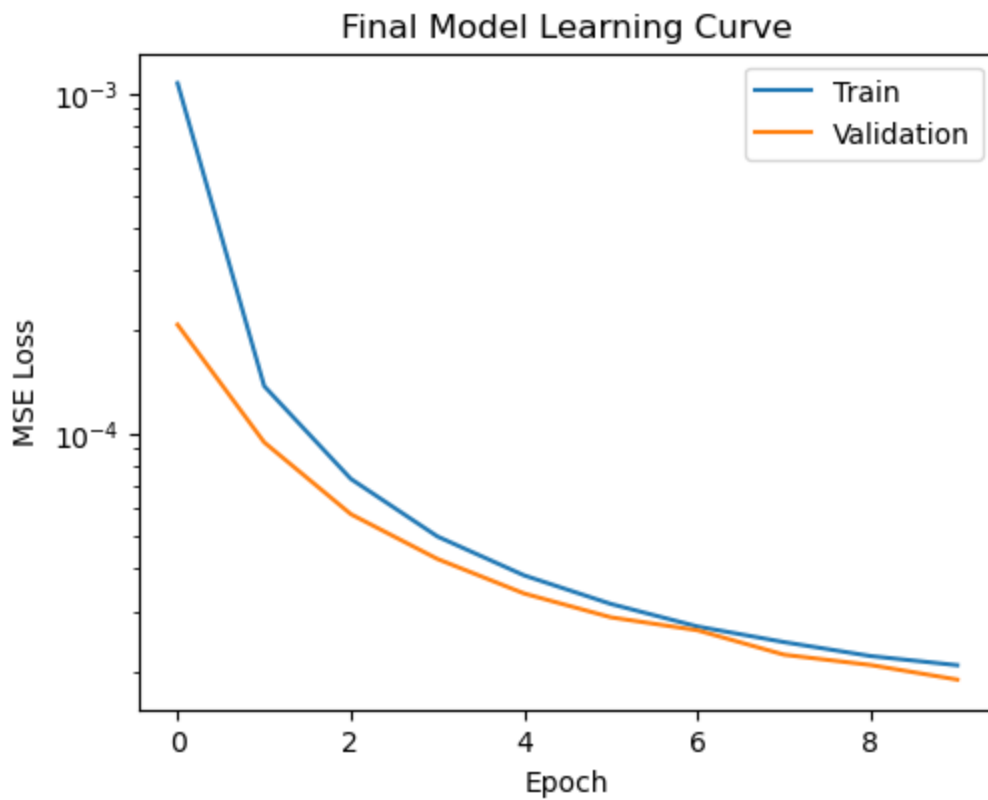
Resulting Validation MAE: 2.1341

--- Testing: Layers=[128], LR=0.0001, Batch=2048 ---

Sequential - Number of parameters: 197636

Initialize training data...

100% |██████████| 10/10 [00:55<00:00, 5.50s/it]



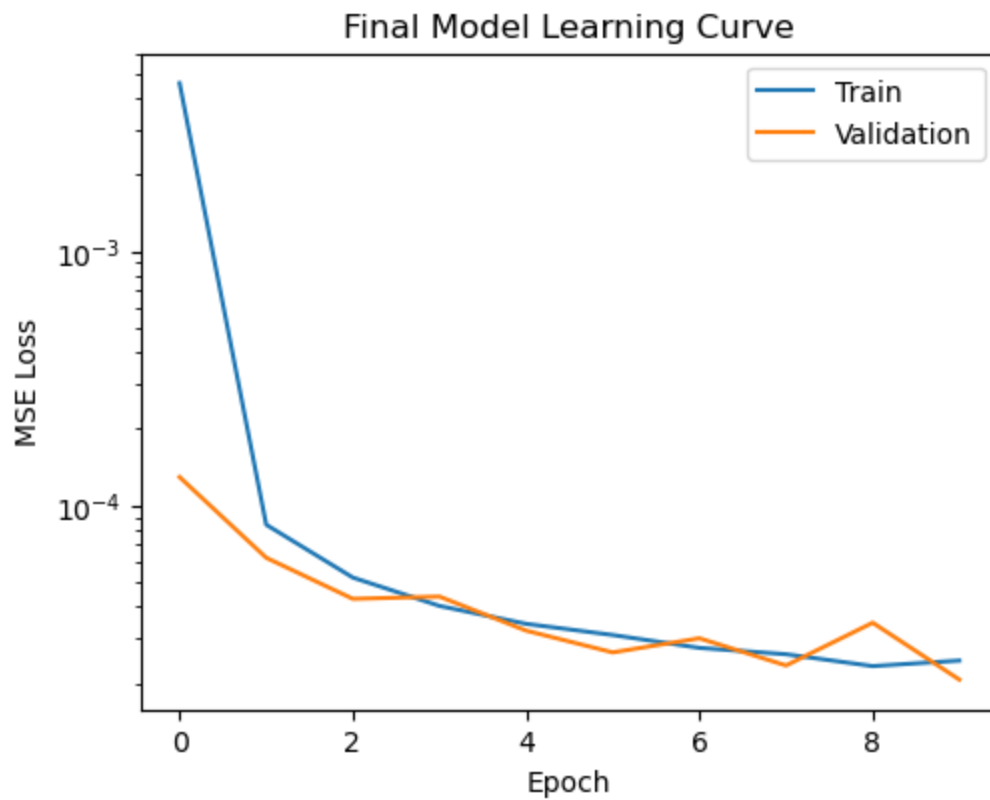
Resulting Validation MAE: 1.7219

--- Testing: Layers=[128, 64], LR=0.001, Batch=4096 ---

Sequential - Number of parameters: 230404

Initialize training data...

100% |██████████| 10/10 [00:50<00:00, 5.01s/it]



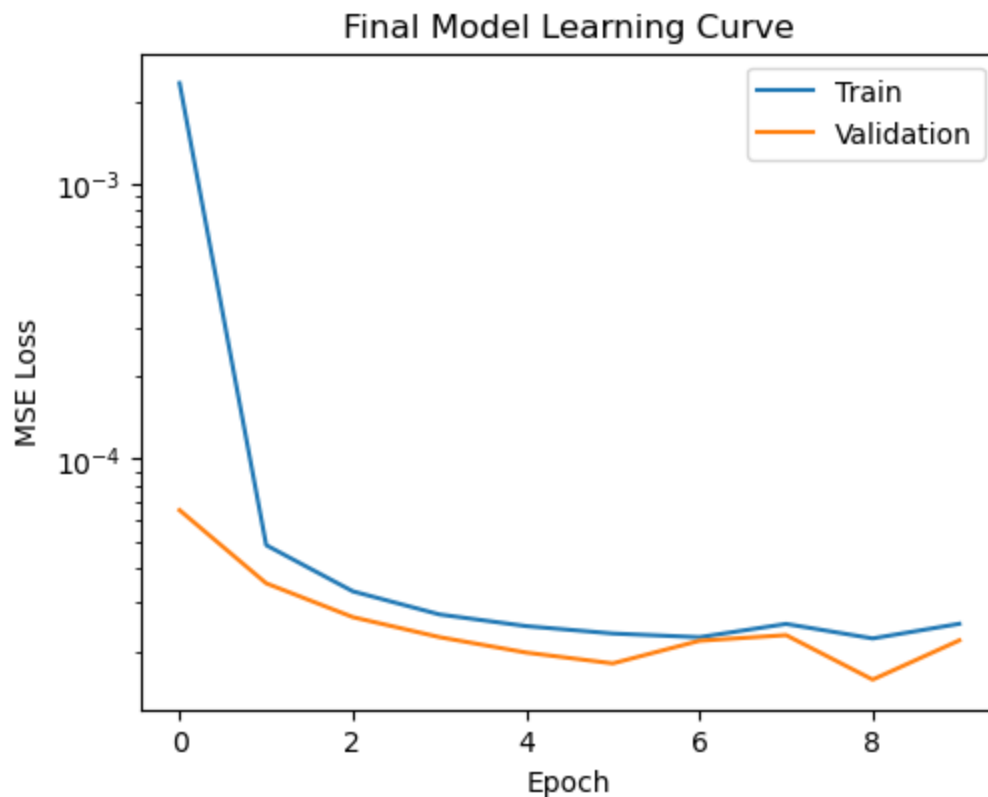
Resulting Validation MAE: 1.8141

--- Testing: Layers=[128, 64], LR=0.001, Batch=2048 ---

Sequential - Number of parameters: 230404

Initialize training data...

100% |██████████| 10/10 [00:57<00:00, 5.78s/it]



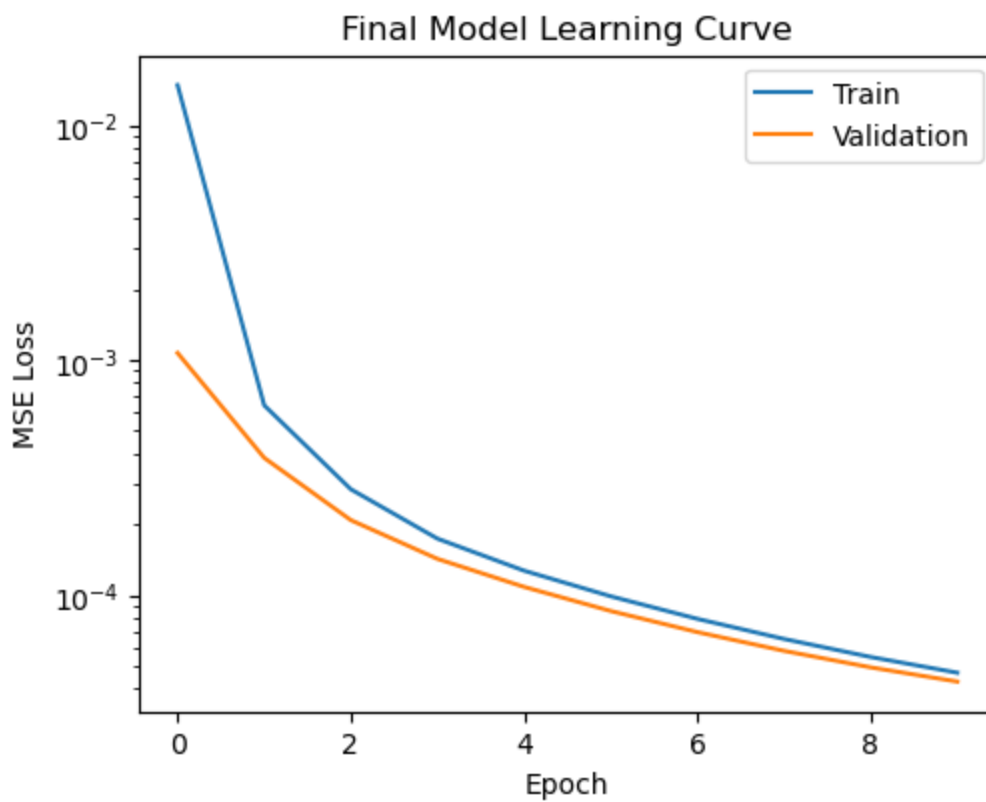
Resulting Validation MAE: 1.6053

--- Testing: Layers=[128, 64], LR=0.0001, Batch=4096 ---

Sequential - Number of parameters: 230404

Initialize training data...

100% |██████████| 10/10 [00:49<00:00, 5.00s/it]



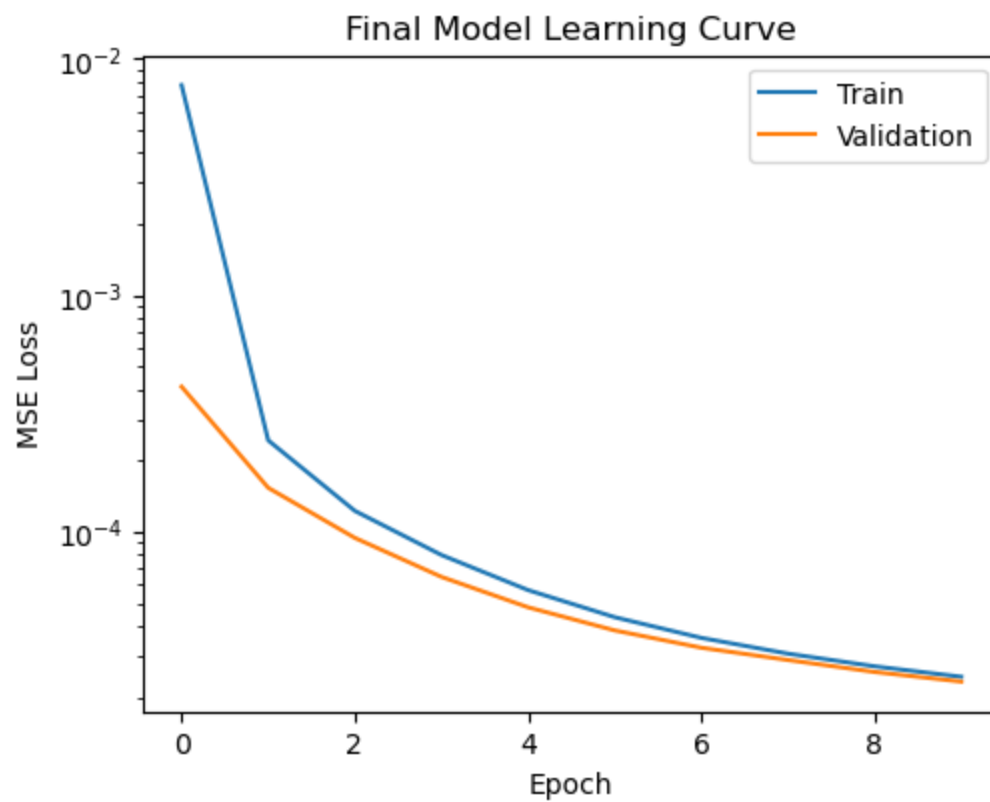
Resulting Validation MAE: 2.2902

--- Testing: Layers=[128, 64], LR=0.0001, Batch=2048 ---

Sequential - Number of parameters: 230404

Initialize training data...

100%|██████████| 10/10 [01:01<00:00, 6.11s/it]



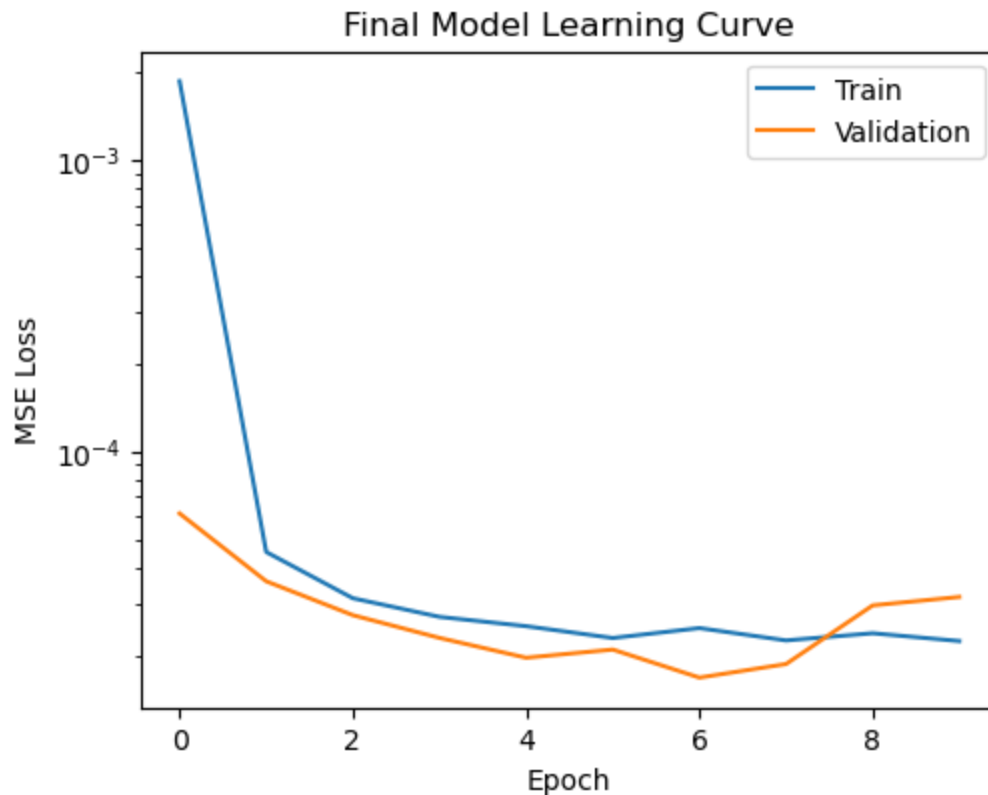
Resulting Validation MAE: 1.8300

--- Testing: Layers=[256, 128], LR=0.001, Batch=4096 ---

Sequential - Number of parameters: 526340

Initialize training data...

100% |██████████| 10/10 [00:51<00:00, 5.16s/it]



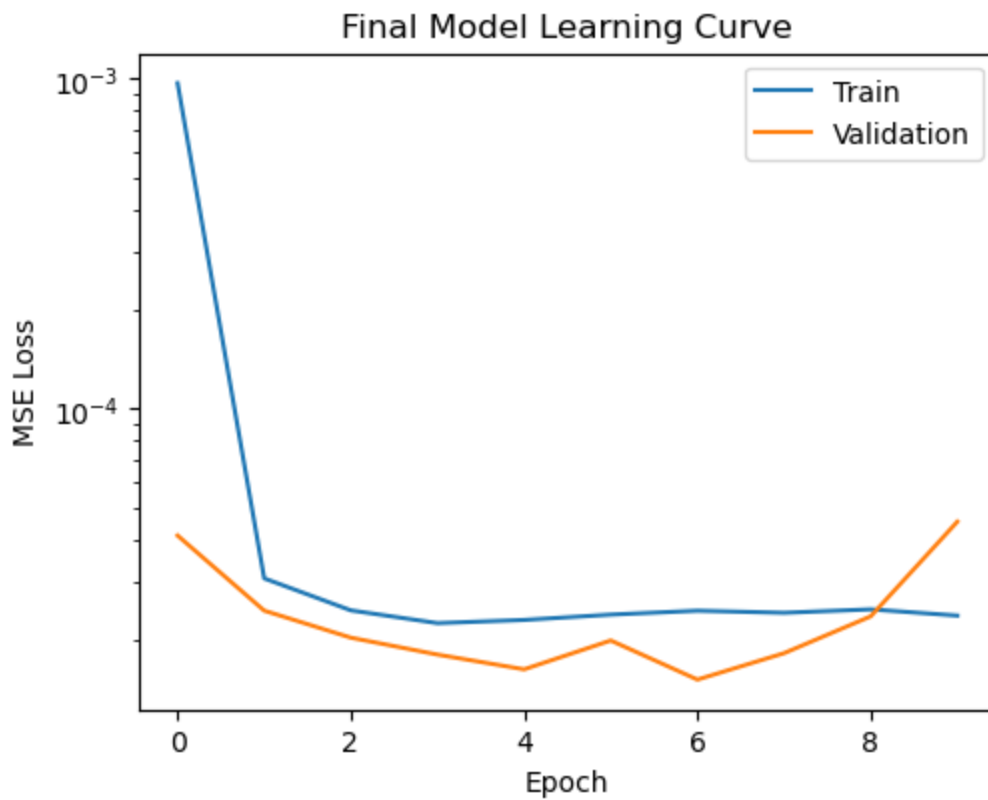
Resulting Validation MAE: 1.6659

--- Testing: Layers=[256, 128], LR=0.001, Batch=2048 ---

Sequential - Number of parameters: 526340

Initialize training data...

100% |██████████| 10/10 [01:02<00:00, 6.27s/it]



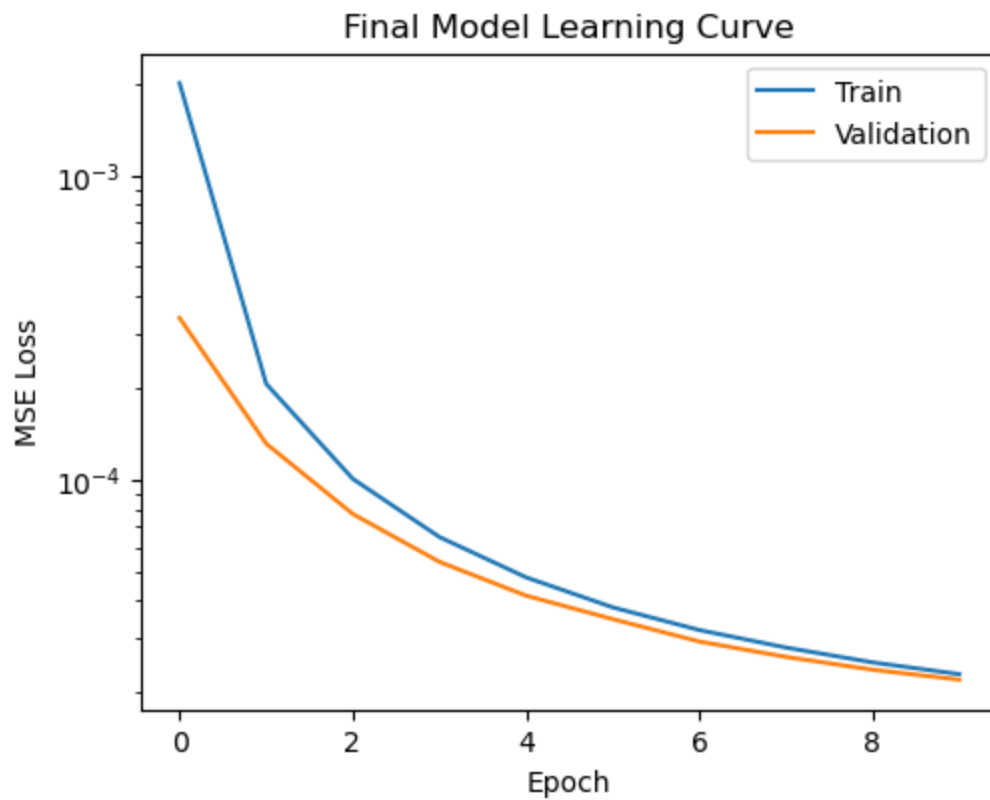
Resulting Validation MAE: 1.6260

--- Testing: Layers=[256, 128], LR=0.0001, Batch=4096 ---

Sequential - Number of parameters: 526340

Initialize training data...

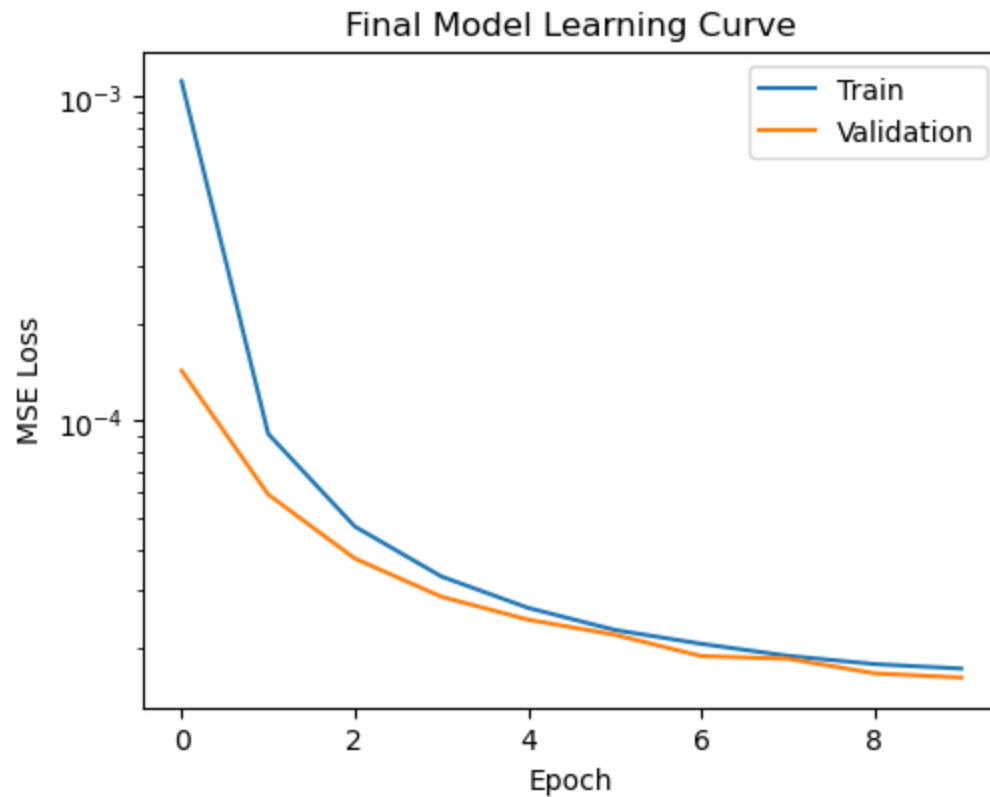
100%|██████████| 10/10 [00:51<00:00, 5.10s/it]



Resulting Validation MAE: 1.7395

--- Testing: Layers=[256, 128], LR=0.0001, Batch=2048 ---
Sequential - Number of parameters: 526340
Initialize training data...

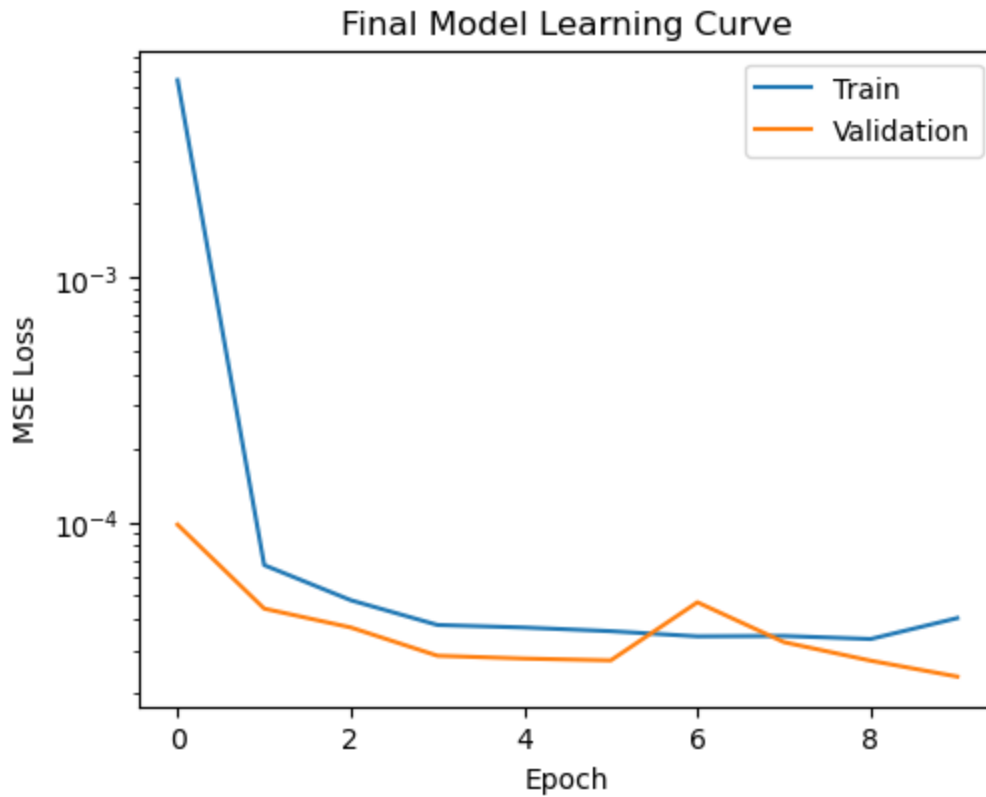
100% |██████████| 10/10 [01:01<00:00, 6.18s/it]



Resulting Validation MAE: 1.5669

--- Testing: Layers=[256, 192, 128, 64], LR=0.001, Batch=4096 ---
Sequential - Number of parameters: 723716
Initialize training data...

100% |██████████| 10/10 [00:58<00:00, 5.81s/it]



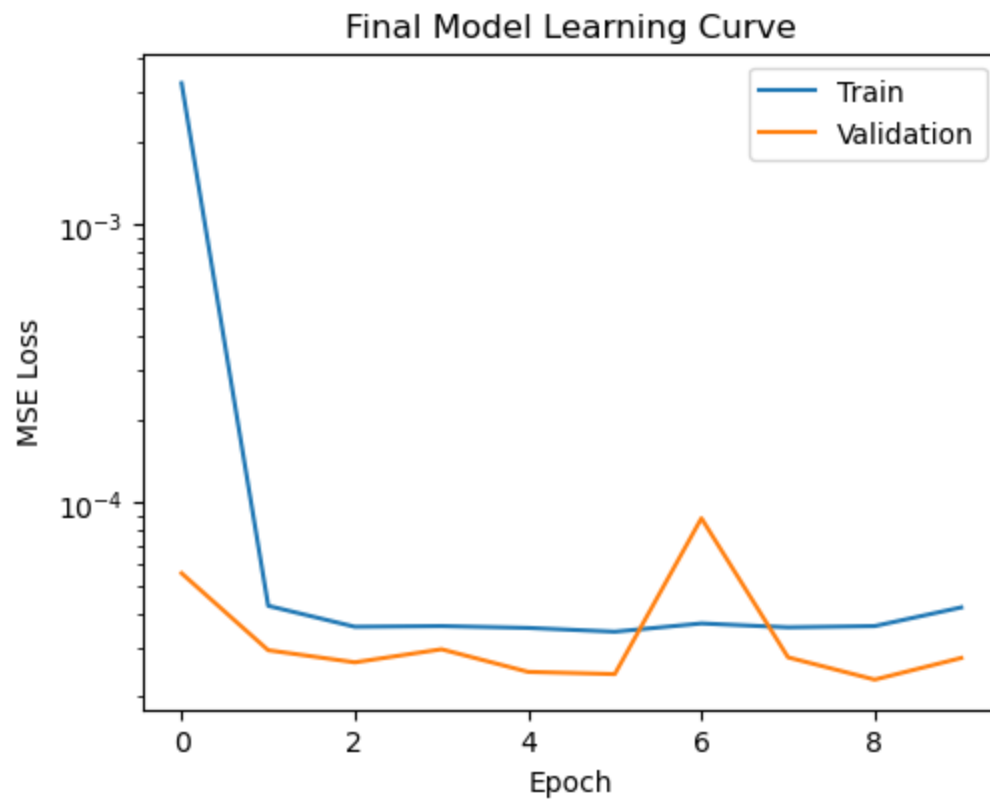
Resulting Validation MAE: 1.9790

--- Testing: Layers=[256, 192, 128, 64], LR=0.001, Batch=2048 ---

Sequential - Number of parameters: 723716

Initialize training data...

100% |██████████| 10/10 [01:12<00:00, 7.21s/it]



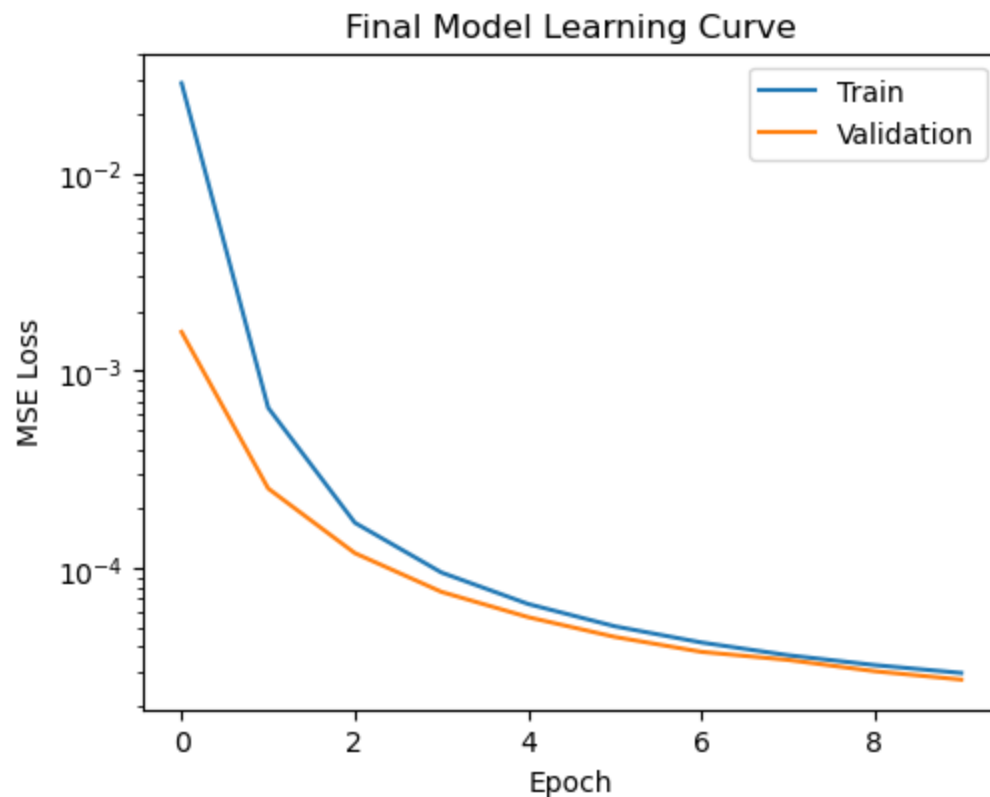
Resulting Validation MAE: 1.9871

--- Testing: Layers=[256, 192, 128, 64], LR=0.0001, Batch=4096 ---

Sequential - Number of parameters: 723716

Initialize training data...

100% |██████████| 10/10 [01:03<00:00, 6.37s/it]



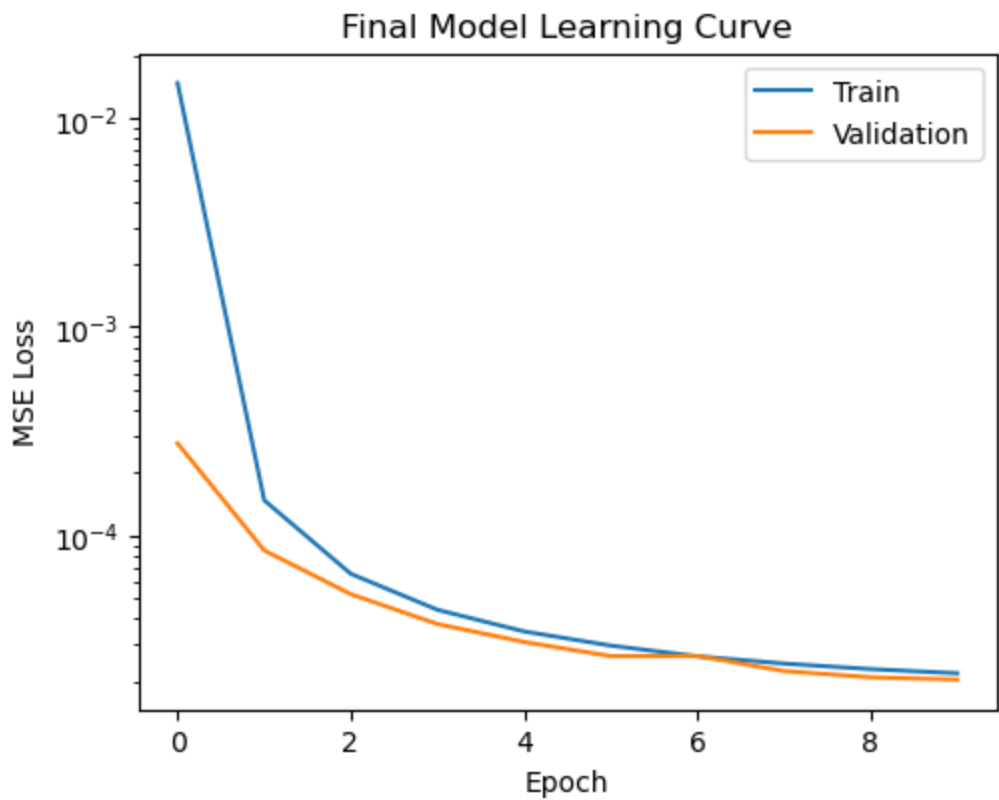
Resulting Validation MAE: 2.0005

--- Testing: Layers=[256, 192, 128, 64], LR=0.0001, Batch=2048 ---

Sequential - Number of parameters: 723716

Initialize training data...

100% |██████████| 10/10 [01:08<00:00, 6.84s/it]



Resulting Validation MAE: 1.8503

```
In [75]: print(f"{'Layers':<25} | {'LR':<7} | {'Batch':<6} | {'RMSE'}")
print("-" * 55)
for r in tuning_results:
    print(f"{r['hidden_layers']:<25} | {r['learning_rate']:<7} | {r['batch_s
```

Layers	LR	Batch	RMSE
[128]	0.001	4096	3.1593
[128]	0.001	2048	2.6679
[128]	0.0001	4096	3.4976
[128]	0.0001	2048	2.7268
[128, 64]	0.001	4096	2.8507
[128, 64]	0.001	2048	2.4922
[128, 64]	0.0001	4096	4.1091
[128, 64]	0.0001	2048	3.0303
[256, 128]	0.001	4096	2.5707
[256, 128]	0.001	2048	2.4451
[256, 128]	0.0001	4096	2.9458
[256, 128]	0.0001	2048	2.5213
[256, 192, 128, 64]	0.001	4096	3.0350
[256, 192, 128, 64]	0.001	2048	3.0156
[256, 192, 128, 64]	0.0001	4096	3.2775
[256, 192, 128, 64]	0.0001	2048	2.8359

```
In [91]: #KEY
# H = 0
# C = 1
# N = 2
# O = 3
```

```

def evaluate_by_heavy_atoms(trainer, data):
    data_loader = data.collate(trainer.batch_size).cache()
    hartree2kcalmol = 627.5094738898777

    errors_by_heavy_atoms = {}

    trainer.model.eval()

    with torch.no_grad():
        for batch in data_loader:
            species = batch["species"].to(device)
            coords = batch["coordinates"].to(device)
            true_energies = batch["energies"].to(device).float()

            _, pred_energies = trainer.model((species, coords))

            errors = torch.abs(true_energies - pred_energies).detach().cpu()
            species_cpu = species.detach().cpu().numpy()

            heavy_atom_counts = (species_cpu != 0).sum(axis=1)

            for count, error in zip(heavy_atom_counts, errors):
                count = int(count)
                if count not in errors_by_heavy_atoms:
                    errors_by_heavy_atoms[count] = []
                errors_by_heavy_atoms[count].append(float(error))

    print("MAE by heavy atom count:")
    heavy_atom_summary = {}

    for count in sorted(errors_by_heavy_atoms.keys()):
        mae = np.mean(errors_by_heavy_atoms[count]) * hartree2kcalmol
        rmse = np.sqrt((np.mean(errors_by_heavy_atoms[count]** 2)) * hartree2kcalmol)
        n = len(errors_by_heavy_atoms[count])
        heavy_atom_summary[count] = {
            "n_samples": n,
            "mae_kcalmol": mae,
            "rmse_kcalmol": rmse
        }
        print(f"{count} heavy atoms: MAE = {mae:.4f}, RMSE = {rmse:.4f}, n = {n}")

    return heavy_atom_summary

heavy_atom_summary = evaluate_by_heavy_atoms(final_trainer, test_data)

```

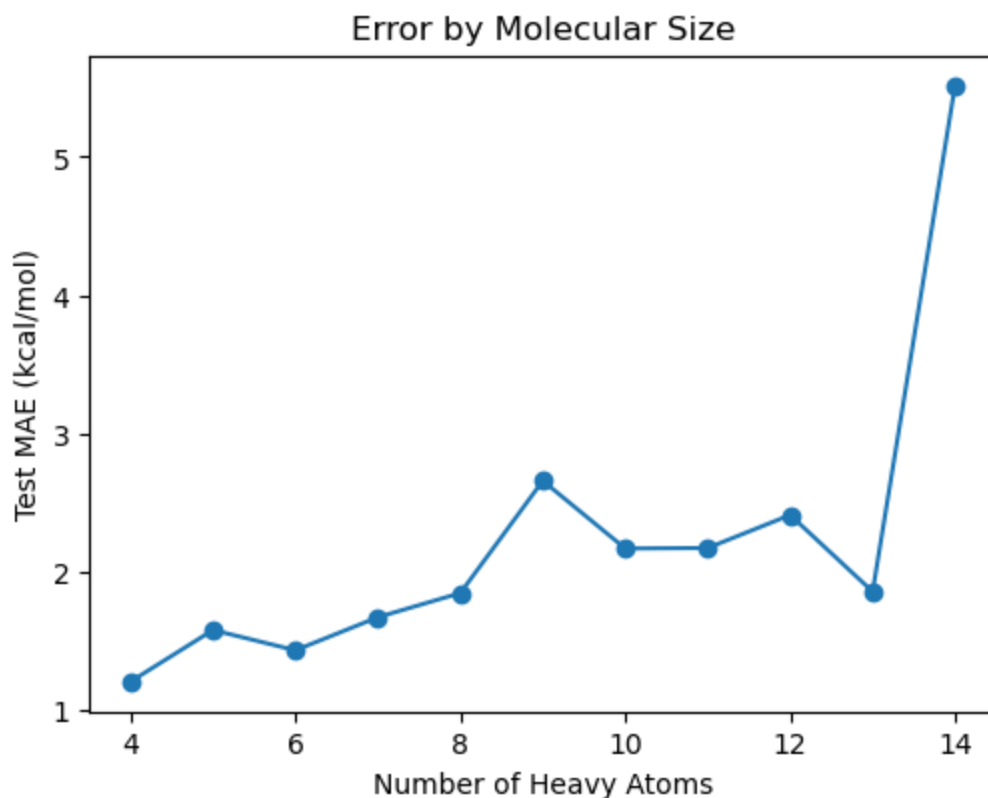
MAE by heavy atom count:

4 heavy atoms: MAE = 1.2011, RMSE = 1.2011, n = 1693
5 heavy atoms: MAE = 1.5746, RMSE = 1.5746, n = 6368
6 heavy atoms: MAE = 1.4278, RMSE = 1.4278, n = 12823
7 heavy atoms: MAE = 1.6672, RMSE = 1.6672, n = 10010
8 heavy atoms: MAE = 1.8423, RMSE = 1.8423, n = 20898
9 heavy atoms: MAE = 2.6552, RMSE = 2.6552, n = 9274
10 heavy atoms: MAE = 2.1650, RMSE = 2.1650, n = 14244
11 heavy atoms: MAE = 2.1686, RMSE = 2.1686, n = 4801
12 heavy atoms: MAE = 2.4075, RMSE = 2.4075, n = 4505
13 heavy atoms: MAE = 1.8527, RMSE = 1.8527, n = 1350
14 heavy atoms: MAE = 5.5200, RMSE = 5.5200, n = 523

MAE heavy atom plot

```
In [93]: counts = sorted(heavy_atom_summary.keys())
maes = [heavy_atom_summary[c]["mae_kcalmol"] for c in counts]

fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
ax.plot(counts, maes, marker="o")
ax.set_xlabel("Number of Heavy Atoms")
ax.set_ylabel("Test MAE (kcal/mol)")
ax.set_title("Error by Molecular Size")
plt.show()
```



Testing Layers=[256, 128], LR=0.0001, Batch=2048

```
In [77]: FINAL_CONFIG2 = {
    "hidden_layers": [256, 128],
    "activation": "relu",
    "batch_size": 2048,
    "learning_rate": 1e-4,
    "epoch": 50,
    "l2": 1e-5,
    "use_batchnorm": True,
    "use_huber": True,
    "grad_clip": 1.0,
    "use_cosine_lr": True,
}

print("Improved final configuration:")
for k, v in FINAL_CONFIG2.items():
    print(f"  {k}: {v}")
```

```
Improved final configuration:
hidden_layers: [256, 128]
activation: relu
batch_size: 2048
learning_rate: 0.0001
epoch: 50
l2: 1e-05
use_batchnorm: True
use_huber: True
grad_clip: 1.0
use_cosine_lr: True
```

```
In [78]: set_seed(42)

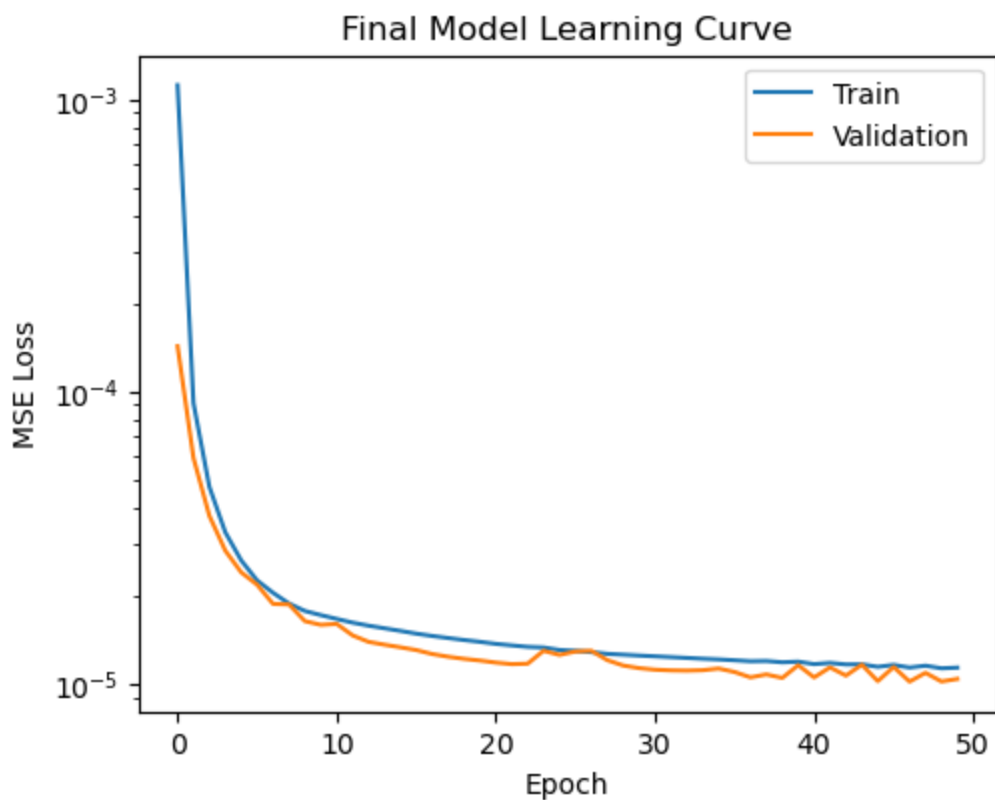
final_model2 = build_model(
    hidden_layers=FINAL_CONFIG2["hidden_layers"],
    activation=FINAL_CONFIG2["activation"]
)

final_trainer2 = ANITrainer(
    model=final_model2,
    batch_size=FINAL_CONFIG2["batch_size"],
    learning_rate=FINAL_CONFIG2["learning_rate"],
    epoch=FINAL_CONFIG2["epoch"],
    l2=FINAL_CONFIG2["l2"]
)

final_train_losses, final_val_losses = final_trainer2.train(
    train_data=train_data,
    val_data=val_data,
    early_stop=True,
    draw_curve=True
)
```

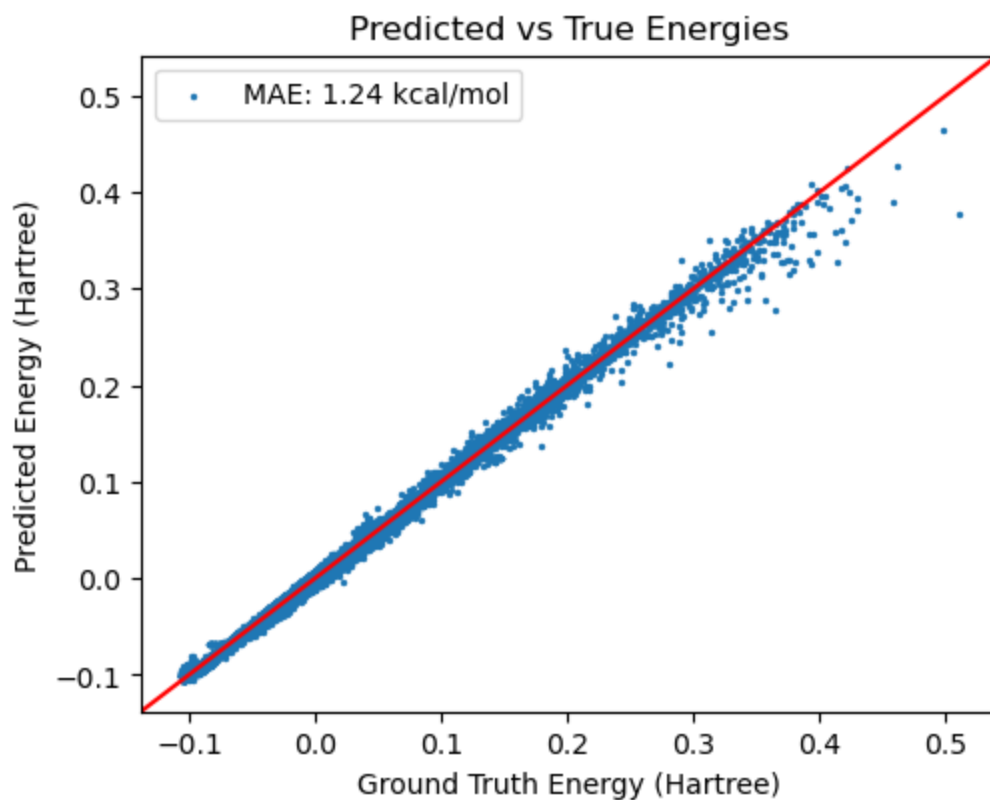
```
Sequential - Number of parameters: 526340
Initialize training data...
```

```
100%|██████████| 50/50 [05:03<00:00, 6.07s/it]
```



```
In [79]: final_test_mse2 = final_trainer2.evaluate(test_data, draw_plot=True)
final_test_mae2 = final_trainer2.evaluate_mae_kcalmol(test_data)
final_test_rmse2 = final_trainer2.evaluate_rmse_kcalmol(test_data)

print("\nFinal model evaluation on held-out test set:")
print(f"Final Test MSE2: {final_test_mse2:.8f}")
print(f"Final Test MAE2: {final_test_mae2:.4f} kcal/mol")
print(f"  Final Test RMSE2: {final_test_rmse2:.4f} kcal/mol")
```



MAE: 1.2397 kcal/mol

Final model evaluation on held-out test set:

Final Test MSE2: 0.00000940

Final Test MAE2: 1.2397 kcal/mol

Final Test RMSE2: 1.9238 kcal/mol

```
In [81]: conclusions2 = {
    "hidden_layers": FINAL_CONFIG2["hidden_layers"],
    "activation": FINAL_CONFIG2["activation"],
    "batch_size": FINAL_CONFIG2["batch_size"],
    "learning_rate": FINAL_CONFIG2["learning_rate"],
    "epochs": FINAL_CONFIG2["epoch"],
    "l2_regularization": FINAL_CONFIG2["l2"],
    "final_test_mse": final_test_mse2,
    "final_test_mae_kcalmol": final_test_mae2
}
print("\nFinal configuration conclusions2/summary:")
for k, v in conclusions2.items():
    print(f"{k}: {v}")
```

Final configuration conclusions2/summary:

hidden_layers: [256, 128]

activation: relu

batch_size: 2048

learning_rate: 0.0001

epochs: 50

l2_regularization: 1e-05

final_test_mse: 9.398918876296946e-06

final_test_mae_kcalmol: 1.239695971364999

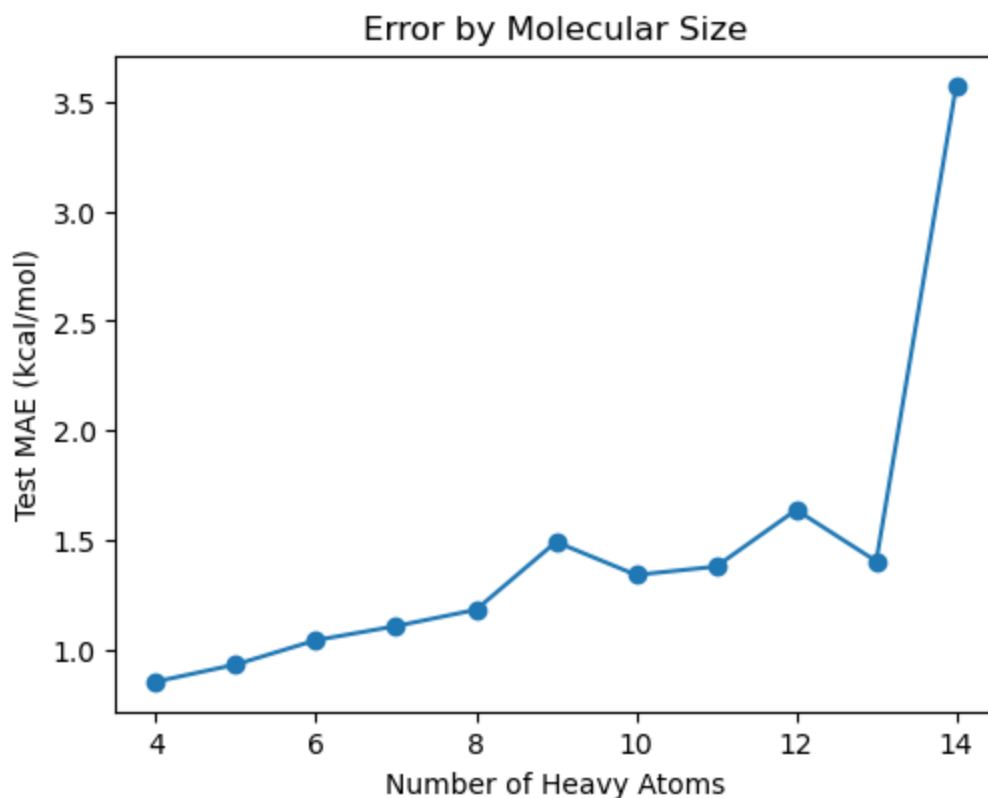
Heavy Atom MAE with Final 2 model

```
In [94]: heavy_atom_summary = evaluate_by_heavy_atoms(final_trainer2, test_data)
counts = sorted(heavy_atom_summary.keys())
maes = [heavy_atom_summary[c]["mae_kcalmol"] for c in counts]

fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
ax.plot(counts, maes, marker="o")
ax.set_xlabel("Number of Heavy Atoms")
ax.set_ylabel("Test MAE (kcal/mol)")
ax.set_title("Error by Molecular Size")
plt.show()
```

MAE by heavy atom count:

4 heavy atoms: MAE = 0.8539, RMSE = 0.8539, n = 1693
5 heavy atoms: MAE = 0.9320, RMSE = 0.9320, n = 6368
6 heavy atoms: MAE = 1.0430, RMSE = 1.0430, n = 12823
7 heavy atoms: MAE = 1.1072, RMSE = 1.1072, n = 10010
8 heavy atoms: MAE = 1.1818, RMSE = 1.1818, n = 20898
9 heavy atoms: MAE = 1.4911, RMSE = 1.4911, n = 9274
10 heavy atoms: MAE = 1.3407, RMSE = 1.3407, n = 14244
11 heavy atoms: MAE = 1.3787, RMSE = 1.3787, n = 4801
12 heavy atoms: MAE = 1.6374, RMSE = 1.6374, n = 4505
13 heavy atoms: MAE = 1.4032, RMSE = 1.4032, n = 1350
14 heavy atoms: MAE = 3.5748, RMSE = 3.5748, n = 523



RMSE Heavy Atom on Final 2 model

```
In [98]: heavy_atom_summary = evaluate_by_heavy_atoms(final_trainer2, test_data)
```

MAE by heavy atom count:

```
4 heavy atoms: MAE = 0.8539, RMSE = 0.8539, n = 1693
5 heavy atoms: MAE = 0.9320, RMSE = 0.9320, n = 6368
6 heavy atoms: MAE = 1.0430, RMSE = 1.0430, n = 12823
7 heavy atoms: MAE = 1.1072, RMSE = 1.1072, n = 10010
8 heavy atoms: MAE = 1.1818, RMSE = 1.1818, n = 20898
9 heavy atoms: MAE = 1.4911, RMSE = 1.4911, n = 9274
10 heavy atoms: MAE = 1.3407, RMSE = 1.3407, n = 14244
11 heavy atoms: MAE = 1.3787, RMSE = 1.3787, n = 4801
12 heavy atoms: MAE = 1.6374, RMSE = 1.6374, n = 4505
13 heavy atoms: MAE = 1.4032, RMSE = 1.4032, n = 1350
14 heavy atoms: MAE = 3.5748, RMSE = 3.5748, n = 523
```

Comparing ANI-1 vs. Model

```
In [99]: paper_train_rmse = 1.2
paper_val_rmse = 1.3
paper_test_rmse = 1.3
paper_transfer_rmse = 1.8

print("\nComparison to ANI-1 paper benchmarks:")
print(f"ANI-1 paper train RMSE: {paper_train_rmse} kcal/mol")
print(f"ANI-1 paper validation RMSE: {paper_val_rmse} kcal/mol")
print(f"ANI-1 paper test RMSE: {paper_test_rmse} kcal/mol")
print(f"ANI-1 transferability benchmark RMSE: {paper_transfer_rmse} kcal/mol")
```

Comparison to ANI-1 paper benchmarks:

ANI-1 paper train RMSE: 1.2 kcal/mol

ANI-1 paper validation RMSE: 1.3 kcal/mol

ANI-1 paper test RMSE: 1.3 kcal/mol

ANI-1 transferability benchmark RMSE: 1.8 kcal/mol

```
In [102]: print("\nCheckpoint 5 complete.")
print(f"Final Test MAE: {final_test_mae2:.4f} kcal/mol")
print(f"Final Test RMSE: {final_test_rmse2:.4f} kcal/mol")
```

Checkpoint 5 complete.

Final Test MAE: 1.2397 kcal/mol

Final Test RMSE: 1.9238 kcal/mol

```
In [101]: heavyten_results = heavy_atom_summary[10]
heavy_8 = heavy_atom_summary[8]
print (heavyten_results)
print (heavy_8)
```

```
{'n_samples': 14244, 'mae_kcalmol': 1.3407116398260583, 'rsme_kcalmol': 1.3407116398260583}
```

```
{'n_samples': 20898, 'mae_kcalmol': 1.1817655782366672, 'rsme_kcalmol': 1.1817655782366672}
```

This project model is smaller and simpler than the full ANI-1 architecture as the final test MAE lies below 2 kcal/mol. Standing only at 1.2397 kcal/mol, it is lower than the ANI-1

paper benchmark of 1.8 kcal/mol, seeing as we run our model on 50 epochs versus the 100 epochs that the ANI-1 original architecture did. Even as molecules get larger (up to 13 heavy atoms), the MAE stays below the 2 kcal/mol line which we see in the "Error by molecule Size" in our tests compared to Figure 4 page 3196, where 10 heavy atoms had a transferability RSME error of 1.8 kcal/mol. Though out RMSE is higher, for up to 13 heavy atoms, our RMSE still ranks below the 1.8 RMSE marker. The largest variability was with 14 heavy atoms.

In []: